



OptiGrid

Energy Intelligence Platform

Architectural Requirements

Version 1.0

July 30, 2026

Team Coreflow

COS 301 - Capstone Project

University of Pretoria

Contents

1	Introduction	2
2	Architectural Requirements	2
2.1	Architectural Patterns	2
2.2	Design Patterns	3
2.3	Constraints	3
2.4	Architectural Diagram	5
2.5	Mapping Quality Requirements to Architectural Decisions	6
3	Technology Requirements	7
4	API Contracts	9
4.1	Authentication	9
4.2	User Access Control	11
4.3	Buildings	14
4.4	Preferences	22
4.5	Analytics	24
4.6	Sensors	25
4.7	Thresholds	28
4.8	Support / Contact	33
4.9	Telemetry	33
4.10	Anomalies	35
5	Deployment	39
5.1	Deployment Requirements	39
5.1.1	Live, Accessible System	39
5.1.2	Environment Parity	39
5.1.3	Infrastructure as Code (IaC) / Containerisation	39
5.1.4	Secrets Management	39
5.1.5	Rollback Strategy	39
5.2	Deployment Diagrams	40
5.2.1	CI/CD Pipeline Diagram	40
6	NFR Traceability Matrix	41
7	Deployment Diagram	44
8	Test Evidence (Screenshots)	45
8.1	Security Tests	45
8.2	Performance Tests	46
8.3	Availability Tests	47
8.4	Reliability Tests	48
8.5	Maintainability Tests	49

1 Introduction

This document outlines the architectural patterns, design choices, system constraints, and the technology stack utilized to keep the platform scalable and secure. The architecture is intentionally designed to support the high throughput of IoT telemetry data while maintaining a responsive and accessible user interface for facility managers.

2 Architectural Requirements

2.1 Architectural Patterns

OptiGrid uses a hybrid approach, combining **Microservices** and **Event-Driven** patterns to ensure the system stays available and scales well under heavy loads.

- **Microservices Architecture:**

- *Definition:* The system is broken down into smaller, independent services based on what they do (like Data Ingestion, Analytics, and the Dashboard UI), rather than existing as a single monolithic application.
- *Justification:* This is necessary because ingesting IoT telemetry data handles significantly more traffic than basic dashboard API queries. Keeping these separate means the ingestion containers can be scaled up during heavy data spikes without paying for extra frontend resources or slowing down the dashboard. It also lets the Python analytics engine run entirely separately from the Node.js backend, allowing for language-specific optimizations.

- **Event-Driven Architecture (EDA):**

- *Definition:* Different parts of the system communicate asynchronously by sending events to a message broker, rather than relying on direct service-to-service calls.
- *Justification:* If thousands of IoT meters try to send data at the exact same time, normal API calls would back up and probably crash the database. Using a Redis queue acts as a buffer. The ingestion service drops incoming data onto the queue, which lets the database write it at a safe, steady pace. This helps prevent data loss even if traffic spikes unexpectedly.

- **Layered Architecture:**

- *Definition:* The system logic is split into four strictly separated presentation, access, services, and database layers. Thus it is a four-tiered layered architecture.
- *Justification:* This keeps the codebase highly organized. For example, changing the user interface will not interfere with how data is ingested at the database level, which makes maintaining and updating the system much easier over time.

2.2 Design Patterns

Several standard design patterns are used in the codebase to keep the system maintainable, modular, and reliable.

- **Model-View-Controller (MVC):**

- *Definition:* Separates the application logic into three interconnected parts to separate internal information from how it is presented to the user.
- *Justification:* This is used extensively in the backend services and loosely mirrored in the React frontend. It separates the raw data models from the dashboard views using controllers. This decoupling makes the code significantly easier to test and keeps different concerns clearly separated during development.

- **Singleton Pattern:**

- *Definition:* Restricts a class so it can only be instantiated once globally.
- *Justification:* This is used for the database connection pools (PostgreSQL and InfluxDB). Creating a brand-new connection for every single request in a busy IoT system would cause massive delays and exhaust the available connections. A Singleton makes sure a single, persistent connection pool is reused efficiently across all incoming requests.

- **Strategy Pattern:**

- *Definition:* A behavioral pattern that lets you select a specific algorithm at runtime from a family of algorithms.
- *Justification:* This is used in the Python Analytics engine to pick the right predictive model. The system relies on Optuna to automatically test and choose the best forecasting strategy based on a building's specific energy profile, rather than hardcoding a single, rigid algorithm that might not fit every scenario.

2.3 Constraints

The architectural choices are shaped by a few strict limitations:

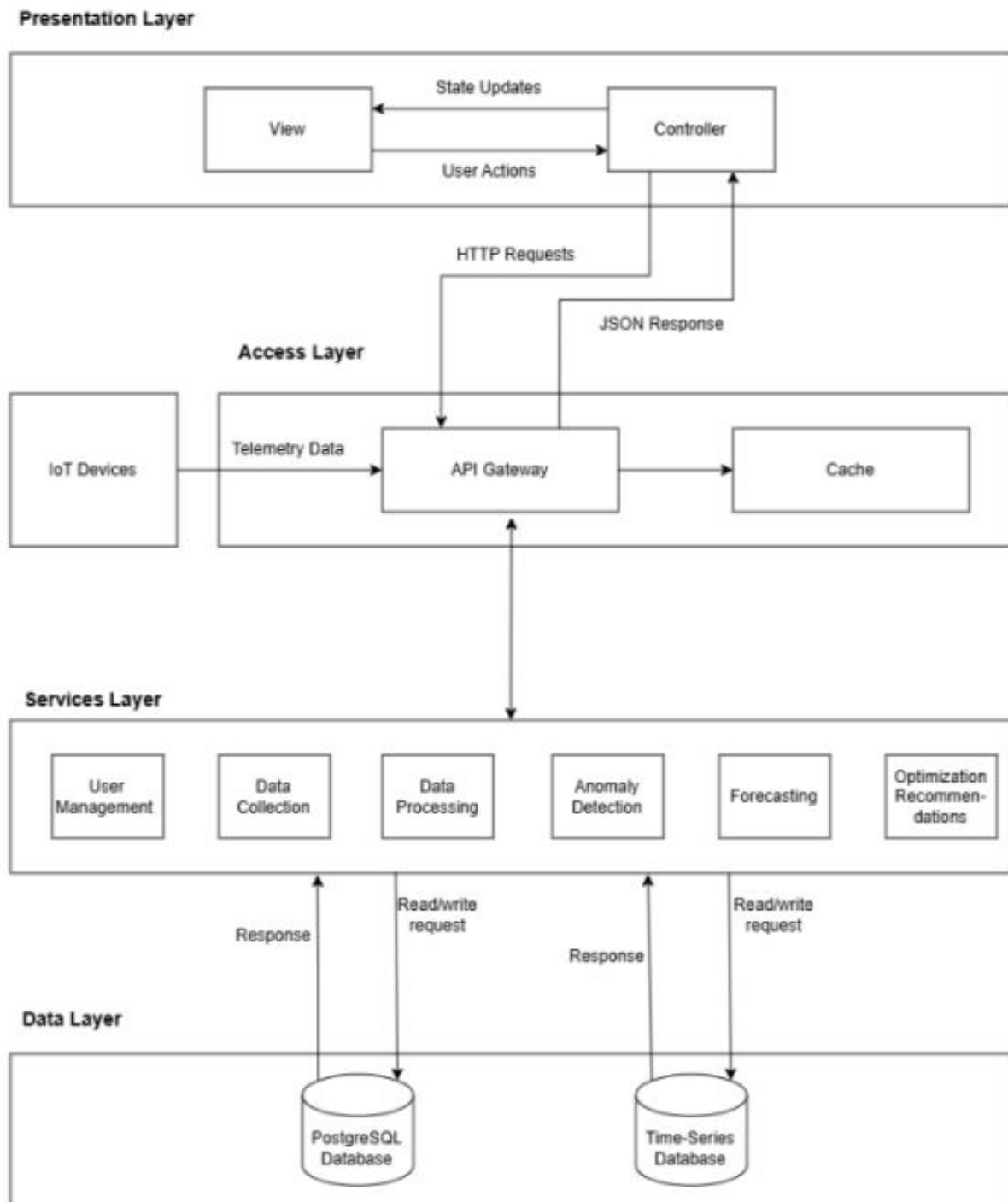
- **Financial & Infrastructure Constraints:** The project operates under a strict R5000 budget provided by EPI-USE. *Impact:* The project cannot use paid enterprise software or expensive proprietary platforms. The system must stick to open-source tools and use cost-effective cloud options such as AWS Free Tier.
- **Hardware & Data Constraints:** Due to a lack of physical access to real-world building smart grids, the system is constrained to supporting simulated telemetry data. *Impact:* The ingestion API has to be flexible enough to take standard JSON payloads over regular HTTPS. It will not be tied to specific hardware protocols, making it easier to integrate real-world sensor hardware later without rewriting the ingestion logic.
- **Deployment Constraints:** The system has to be fully containerised and deployed through an automated CI/CD pipeline, with no manual interventions allowed. *Im-*

pact: Docker must be used for all services. All configuration settings are handled dynamically through `.env` variables so that development, staging, and production environments are the same without needing to alter source code.

- **Latency & Performance Constraints:** Real-time dashboards must reflect data updates in under two seconds. *Impact:* A regular relational database cannot handle the heavy write speeds and fast read queries needed for IoT data. The system is therefore required to use a dedicated time-series database, InfluxDB.
- **Security & Regulatory Constraints:** The system handles potentially sensitive building operational data and must adhere strictly to the Protection of Personal Information Act (POPIA). *Impact:* Data has to be encrypted when stored, and strict Role-Based Access Control (RBAC) is required so users only see their own buildings. The system must also keep comprehensive audit logs.

2.4 Architectural Diagram

Below is the technology-neutral structural blueprint of OptiGrid, illustrating the separation of concerns and data flow.



2.5 Mapping Quality Requirements to Architectural Decisions

This table maps the quantified Non-Functional Requirements from the SRS directly to the architectural mechanisms that guarantee their fulfilment.

Quality Requirement (from SRS)	Architectural Mechanism & Justification
Performance: Respond to requests within 2 seconds for 95% of requests.	Time-Series Database & Pre-aggregation: Using InfluxDB for telemetry makes querying date ranges very fast. Background cron jobs are also used to pre-calculate data into 15-minute and hourly summaries. This means the dashboard only has to load small datasets instead of calculating millions of rows on the fly, keeping load times under a second.
Security: AES-256 encryption and multi-factor authentication enforcement.	Centralised Authentication + TLS 1.3 + JWT: The authentication layer serves as a strict gateway. All outside traffic is forced to use HTTPS (TLS 1.3), and short-lived JSON Web Tokens are used to verify user permissions on every request without storing session data on the server.
Reliability: 99.9% uptime and recovery from critical failures in less than five minutes.	Message Broker (Redis Queue) Buffering: If the main database goes down, the ingestion API will not lose data. Incoming telemetry waits safely in the Redis queue until the database comes back online, preventing data loss while the system automatically recovers.
Usability: Core tasks completed in less than five minutes, with 85% user satisfaction.	Dashboard-First MVC UI Design: Using React along with a robust state management system allows the UI to update instantly without reloading the page. A standard set of design tokens keeps navigation clear and contrast ratios accessible.
Maintainability: New features or bug fixes deployable within two hours, with 80% test coverage.	Automated Pipelines & Separation of Concerns: Because the layers are strictly separated and MVC is used, the code is easier to unit test. The CI/CD pipeline runs these tests automatically, ensuring code is not merged unless it meets the 80% coverage rule.

3 Technology Requirements

The following technology stack was selected to balance handling lots of data, running machine-learning predictions, and building the UI quickly.

Component	Selected Technology	Justification
Frontend	React.js	React is suitable for building interactive, data-heavy dashboards. Its virtual DOM is fast enough to update complex graphs in real time without lagging. It also has a large community, making it easy to integrate charting libraries such as Recharts or Chart.js.
Backend API	Node.js (Express)	The main challenge for OptiGrid’s backend is handling thousands of small API requests at the same time. Node.js uses an event loop that does not block other processes, making it well suited to high volumes of concurrent connections.
Relational Database	PostgreSQL (Supabase)	This is used for system metadata such as user profiles, building configurations, and permissions. PostgreSQL is reliable, handles complex table joins well, and ensures data remains isolated between tenants.
Time-Series Database	InfluxDB	Trying to store millions of telemetry points in a standard SQL database can degrade indexing performance. InfluxDB is designed for time-stamped data, allowing fast writes and automatic downsampling.
Analytics Engine	Python (scikit-learn, Pandas, Optuna)	Python is a standard choice for data science. Libraries such as scikit-learn, Pandas, and Optuna allow the system to predict future demand based on historical data. Placing this functionality in a separate microservice avoids bloating the Node.js backend.
Containerisation	Docker	Docker solves the “it works on my machine” problem. It ensures the code runs consistently on developers’ computers, in development environments, and on AWS.

Component	Selected Technology	Justification
Hosting	AWS (Backend), Vercel (Frontend)	AWS hosts the backend services and supports Docker-based deployment. Vercel hosts the frontend and communicates with AWS when data is requested. Vercel also creates pre-production environments so changes can be reviewed before being promoted to production.

4 API Contracts

4.1 Authentication

1. signup

- POST

- Request:

```
{
  email: string,
  password: string,
  name: string
}
```

Example:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "name": "John Doe"
}
```

- Response: (201 Created)

```
{
  "message": "User created successfully",
  "user": {
    "userId": "123e4567-e89b-12d3-a456-426614174000",
    "email": "user@example.com",
    "firstName": "John",
    "lastName": "Doe"
  }
}
```

- Error Responses:

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalid email or password format"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- Effect:

— Creates a new user.

2. login

- POST

- Request:

```
{
  email: string,
  password: string
}
```

Example:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!"
}
```

- Response: (200 OK)

```
{
  "message": "Login successful",
  "user": {
    "userId": "123e4567-e89b-12d3-a456-426614174000",
    "email": "user@example.com",
    "firstName": "John",
    "lastName": "Doe",
    "token": "eyJhbGciOiJIUzI1NiIsInR..."
  }
}
```

- Error Responses:

401 Unauthorized

```
{
  "error_code": "UNAUTHORIZED",
  "message": "Invalid credentials provided"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- Effect:

- Authenticates a user using their email and password.
- Returns user information together with a JWT token that expires after a configured period.

4.2 User Access Control

1. getViewers

- GET
- Request:None

Example:

GET /auth/viewers

- Response: (200 OK)

```
{
  "data": [
    {
      "userId": "123e4567-e89b-12d3-a456-426614174000",
      "email": "viewer@example.com",
      "firstName": "Jane",
      "lastName": "Smith",
      "roleType": "VIEWER",
      "buildingIds": ["uuid-1", "uuid-2"]
    }
  ]
}
```

- Error Responses:

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- Effect:
 - Fetches all users with the VIEWER role and their assigned building IDs.
 - Requires ADMIN access.

2. getManagers

- GET
- Request:None

Example:

GET /auth/managers

- **Response:** (200 OK)

```
{
  "data": [
    {
      "userId": "123e4567-e89b-12d3-a456-426614174000",
      "email": "manager@example.com",
      "firstName": "Mark",
      "lastName": "Johnson",
      "roleType": "BUILDING_MANAGER",
      "buildingIds": ["uuid-1"]
    }
  ]
}
```

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Fetches all users with the BUILDING_MANAGER role and their assigned building IDs.
- Requires ADMIN access.

3. assignManager

- POST

- **Request:**

```
{
  userId: string,
  buildingId: string
}
```

Example:

```
{
  "userId": "123e4567-e89b-12d3-a456-426614174000",
  "buildingId": "888e4567-e89b-12d3-a456-426614174000"
}
```

- **Response:** (200 OK)

```
{
  "success": true,
  "message": "Manager assigned to building successfully"
}
```

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalidrequestbody"
}
```

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Assigns a building manager to a specific building.
- Requires ADMIN access.

4. removeManager

- DELETE

- **Request:**

```
{
  userId: string,
  buildingId: string
}
```

Example:

```
{
  "userId": "123e4567-e89b-12d3-a456-426614174000",
  "buildingId": "888e4567-e89b-12d3-a456-426614174000"
}
```

- **Response:** (200 OK)

```
{
```

```

    "success": true,
    "message": "Manager removed from building successfully"
}

```

- **Error Responses:**

403 Forbidden

```

{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}

```

404 Not Found

```

{
  "error_code": "NOT_FOUND",
  "message": "Manager or building not found"
}

```

500 Internal Server Error

```

{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}

```

- **Effect:**

- Removes a manager's access from a specific building.
- Requires ADMIN access.

4.3 Buildings

1. getAllBuildings

- GET

- **Request:**

Query Parameters:

lifecycle_state: string (optional)

Example:

GET /admin?lifecycle_state=ACTIVE

- **Response: (200 OK)**

```

{
  "status": "success",
  "data": [
    {
      "building_id": "123e4567-e89b-12d3-a456-426614174000",
      "building_name": "Main Campus",
      "building_type": "EDUCATIONAL",

```

```

        "lifecycle_state": "ACTIVE"
    }
]
}

```

- **Error Responses:**

403 Forbidden

```

{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}

```

500 Internal Server Error

```

{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}

```

- **Effect:**

- Fetches all buildings that are in the database.
- Requires ADMIN access.

2. getManagerBuildings

- GET

- **Request:**None

Example:

GET /manager

- **Response:** (200 OK)

```

{
  "status": "success",
  "data": [
    {
      "building_id": "123e4567-e89b-12d3-a456-426614174000",
      "building_name": "Science Block",
      "todays_usage": 152.4
    }
  ]
}

```

- **Error Responses:**

403 Forbidden

```

{
  "error_code": "FORBIDDEN",

```

```
    "message": "User does not have access"
  }
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Fetches all buildings assigned to the currently authenticated manager.

3. createBuilding

- **POST**

- **Request:**

Headers:

```
    idempotency-key: string
```

Body:

```
{
  building_name: string,
  building_type: string,
  square_footage?: number,
  timezone?: string,
  max_occupancy?: number,
  physical_address?: string,
  latitude?: number,
  longitude?: number,
  geohash?: string
}
```

Example:

POST /api/buildings

Headers: {

```
    "idempotency-key": "550e8400-e29b-41d4-a716-446655440000"
```

}

{

```
    "building_name": "Corporate Headquarters",
```

```
    "building_type": "OFFICE",
```

```
    "square_footage": 75000,
```

```
    "physical_address": "123 Main Street"
```

}

- **Response: (201 Created)**

```
{
```

```
    "status": "success",
```

```
    "data": {
```

```
        "id": "123e4567-e89b-12d3-a456-426614174000",
```

```

        "building_name": "Corporate Headquarters",
        "building_type": "OFFICE"
    }
}

```

- **Error Responses:**

400 Bad Request

```

{
  "error_code": "INVALID_INPUT",
  "message": "Invalid request body"
}

```

403 Forbidden

```

{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}

```

500 Internal Server Error

```

{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}

```

- **Effect:**

- Creates a new building with the provided details.
- Uses an idempotency key to prevent duplicate requests.
- Creates the building in the PROVISIONING state by default.

4. `getBuildingDetails`

- GET

- **Request:**

Path Parameters:

```
building_id: string
```

Example:

```
GET /api/buildings/123e4567-e89b-12d3-a456-426614174000
```

- **Response: (200 OK)**

```

{
  "status": "success",
  "data": {
    "building_id": "123e4567-e89b-12d3-a456-426614174000",
    "building_name": "Corporate Headquarters",
    "timezone": "Africa/Johannesburg"
  }
}

```

```
    }  
  }
```

- **Error Responses:**

- 403 Forbidden**

```
{  
  "error_code": "FORBIDDEN",  
  "message": "User does not have access"  
}
```

- 404 Not Found**

```
{  
  "error_code": "NOT_FOUND",  
  "message": "Building not found"  
}
```

- 500 Internal Server Error**

```
{  
  "error_code": "SERVER_ERROR",  
  "message": "Internal server error"  
}
```

- **Effect:**

- Returns the relevant details for a single building.
 - Is only accessible when the authenticated user has access to the building.

5. **getBuildingEnergyConsumption**

- **GET**

- **Request:**

Path Parameters:

building_id: string

Query Parameters:

time_range: string (e.g., "7d", "30d")

Example:

GET /api/buildings/.../energy-consumption?time_range=7d

- **Response: (200 OK)**

```
{  
  "status": "success",  
  "data": {  
    "building_id": "123e4567-e89b-12d3-a456-426614174000",  
    "total_kwh": 1234.56,  
    "average_daily_kwh": 41.15,  
    "cost_per_kwh": 2.5  
  }  
}
```

```
}
```

- **Error Responses:**

- 400 Bad Request**

```
{
  "error_code": "INVALIDQUERY",
  "message": "Invalidtime_rangespecified"
}
```

- 403 Forbidden**

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

- 404 Not Found**

```
{
  "error_code": "NOT_FOUND",
  "message": "Buildingnotfound"
}
```

- 500 Internal Server Error**

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Returns energy-consumption metrics for a single building over a selected time range.

6. updateBuilding

- PATCH

- **Request:**

Path Parameters:

building_id: string

Body:

```
{
  building_name?: string,
  building_type?: string,
  square_footage?: number,
  timezone?: string,
  max_occupancy?: number,
  physical_address?: string
}
```

Example:

```
PATCH /api/buildings/123e4567-e89b-12d3-a456-426614174000
{
  "building_name": "Renamed Corporate Office"
}
```

- **Response:** (200 OK)

```
{
  "status": "success",
  "data": { ...updated_building_details... }
}
```

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalidrequestbody"
}
```

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Buildingnotfound"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Updates specific properties of a building.
- Requires ADMIN or MANAGER access.

7. deleteBuilding

- DELETE

- **Request:**

Path Parameters:

```
building_id: string
```

Example:

DELETE /api/buildings/123e4567-e89b-12d3-a456-426614174000

- **Response:** (200 OK)

```
{
  "status": "success",
  "message": "Building successfully deleted"
}
```

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Buildingnotfound"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Removes a building from the database.
- Requires ADMIN access.

8. compareBuildings

- POST

- **Request:**

Headers:

idempotency-key: string

Query Parameters:

buildingID_1: string

buildingID_2: string

time_range: string (e.g., "30d")

Example:

POST /api/buildings/compare

?buildingID_1=uuid1

```
&buildingID_2=uuid2
&time_range=30d
```

- **Response:** (200 OK)

```
{
  "status": "success",
  "data": {
    "building_a": {
      "energy_consumption": 1500,
      "cost": 3000
    },
    "building_b": {
      "energy_consumption": 1200,
      "cost": 2400
    },
    "comparison": { ... }
  }
}
```

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalidqueryparameters"
}
```

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Compares energy consumption and performance metrics between two chosen buildings for a specified time range.

4.4 Preferences

1. **getUserTheme**

- GET

- **Request:**None

Example:

GET /api/preferences/theme

- **Response:** (200 OK)

```
{
  "theme": "dark"
}
```

- **Error Responses:**

401 Unauthorized

```
{
  "error_code": "UNAUTHORIZED",
  "message": "Authentication required"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

– Retrieves the current user's theme preference: light, dark, or system.

2. updateUserTheme

- PUT

- **Request:**

```
{
  theme: "light" | "dark" | "system"
}
```

Example:

PUT /api/preferences/theme

- **Response:** (200 OK)

Preference updated.

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalid theme preference"
}
```

401 Unauthorized

```
{
  "error_code": "UNAUTHORIZED",
  "message": "Authentication required"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Updates the current user's theme preference in the database.

4.5 Analytics

1. getForecast

- POST

- **Request:**

Path Parameters:

building_id: string

Body:

```
{
  horizon_days: number,
  granularity: string
}
```

Example:

POST /api/analytics/forecast/...

- **Response:** (200 OK)

```
{
  "historical": [ ... ],
  "forecast": [ ... ],
  "summary": { ... }
}
```

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalid request body"
}
```

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Building not found"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Calculates an energy-usage forecast graph for a specific building based on historical data.

4.6 Sensors

1. handleSensorTelemetry

- POST

- **Request:**

```
{
  building_id: string,
  sensor_id: string,
  usage: number,
  timestamp?: string
}
```

- **Response:** (201 Created)

```
{
  "status": "success"
}
```

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalid request body"
}
```

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Submits telemetry data from sensors securely.

2. listSensors

- GET

- **Request:**

Query Parameters:

building_id: string

Example:

GET /api/sensors?building_id=...

- **Response: (200 OK)**

```
[
  {
    "id": "sensor-001",
    "mac_address": "AA:BB:CC:DD:EE:FF",
    "status": "Active"
  }
]
```

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Building not found"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Returns every sensor registered to the specified building.
- Is allowed for any authenticated user with access to the building.

3. registerSensor

- POST

- **Request:**

```
{
  building_id: string,
  mac_address: string,
  sensor_type: string,
  unit: string,
  location_zone: string,
  status: string
}
```

- **Response:** (201 Created)

Sensor registered successfully.

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalidrequestbody"
}
```

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Registers a new sensor for the specified building.

- Requires ADMIN or MANAGER access.

4. deleteSensor

- DELETE

- Request:

Path Parameters:

`sensor_id: string`

Example:

DELETE /api/sensors/sensor-001

- Response: (200 OK)

Sensor deleted successfully.

- Error Responses:

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Sensornotfound"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- Effect:

- Deletes the sensor from its building.
- Requires ADMIN or MANAGER access.

4.7 Thresholds

1. createThreshold

- POST

- Request:

POST /api/thresholds

```
{
  "building_id": string,
```

```

    "metric_type": string,
    "unit": string,
    "upper_limit_kw": number,
    "lower_limit_kw": number,
    "allowed_spike_percentage": number,
    "use_z_score": boolean,
    "z_score_threshold": number
}

```

- **Response:** (201 Created)

```

{
  "status": "success",
  "data": {
    "threshold_id": string,
    "building_id": string
  }
}

```

- **Error Responses:**

403 Forbidden

```

{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}

```

500 Internal Server Error

```

{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}

```

- **Effect:**

- Creates an alert threshold for a building.

2. getThresholds

- GET

- **Request:**

Path Parameters:

```

    buildingId: string

```

```

GET /api/thresholds/building/{buildingId}

```

- **Response:** (200 OK)

```

{
  "status": "success",
  "data": [

```

```

        {
            "threshold_id": string,
            "metric_type": string,
            "upper_limit_kw": number
        }
    ]
}

```

- **Error Responses:**

403 Forbidden

```

{
    "error_code": "FORBIDDEN",
    "message": "User does not have access"
}

```

500 Internal Server Error

```

{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}

```

- **Effect:**

- Fetches the active thresholds related to a specific building.

3. updateThreshold

- PATCH

- **Request:**

Path Parameters:

id: string

PATCH /api/thresholds/{id}

```

{
    "upper_limit_kw": number,
    "lower_limit_kw": number,
    "allowed_spike_percentage": number,
    "use_z_score": boolean,
    "z_score_threshold": number,
    "is_active": boolean,
    "muted_until": string (date-time)
}

```

- **Response: (200 OK)**

```

{
    "status": "success",
    "data": { ... }
}

```

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Threshold not found"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Modifies selected fields of a threshold, such as its upper limit or z-score configuration.

4. deleteThreshold

- DELETE

- **Request:**

Path Parameters:

id: string

DELETE /api/thresholds/{id}

- **Response:** (200 OK)

```
{
  "status": "success",
  "message": "Threshold deleted"
}
```

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Thresholdnotfound"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Removes a threshold and synchronises the changes.

5. muteThreshold

- PATCH

- **Request:**

Path Parameters:

id: string

```
PATCH /api/thresholds/{id}/mute
{
  "muted_until": string (date-time)
}
```

- **Response: (200 OK)**

```
{
  "status": "success",
  "data": {
    "muted_until": string (date-time)
  }
}
```

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Thresholdnotfound"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Sets or clears a threshold's muting period so alerts are not generated.

4.8 Support / Contact

1. handleSubmit

- POST

- **Request:**

```
POST /api/contact
{
  "inquiryType": "Building",
  "subject": "Cannot create a building",
  "message": "I cannot create a building and need help."
}
```

- **Response:** (200 OK)

```
{
  "success": true,
  "message": "Ticket Received",
  "id": "5b304c4f-1234-5f56-4321-8dcb76cd1123"
}
```

- **Error Responses:**

```
400 Bad Request
{
  "success": false,
  "error": "Failed to send: API key is missing or invalid"
}
```

- **Effect:**

- Sends a contact or support-request email through Resend.

4.9 Telemetry

1. ingestTelemetry

- POST

- **Request:**

```
POST /api/telemetry/ingest
```

- **Response:** (200 OK)

- **Error Responses:**

400 Bad Request

```
{
  "error_code": "INVALID_INPUT",
  "message": "Invalidtelemetrypayload"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Is used by physical circuits and the Python emulator to ingest data.

2. `getLivePortfolioTelemetry`

- GET

- **Request:**

```
GET /api/telemetry/live
```

- **Response:** (200 OK)

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internalservererror"
}
```

- **Effect:**

- Is used by the frontend real-time dashboard for aggregate views.

3. `streamTelemetry`

- GET

- **Request:**

```
Path Parameters:
  building_id: string
```

GET /api/telemetry/stream/{building_id}

- **Response:** (200 OK)

Stream response.

- **Error Responses:**

403 Forbidden

```
{
  "error_code": "FORBIDDEN",
  "message": "User does not have access"
}
```

404 Not Found

```
{
  "error_code": "NOT_FOUND",
  "message": "Building not found"
}
```

500 Internal Server Error

```
{
  "error_code": "SERVER_ERROR",
  "message": "Internal server error"
}
```

- **Effect:**

- Is used by the frontend dashboard to stream telemetry.

4.10 Anomalies

1. getAnomalies

- GET

- **Request:**

Path Parameters:

buildingId: string

Query Parameters:

skip, take, severity, status, startDate, endDate

GET /api/anomalies/building/{buildingId}

- **Response:** (200 OK)

```
{
  "status": "success",
  "data": [
    {
      "anomaly_id": string,

```

```

        "severity_level": string,
        "status": string,
        "detected_timestamp": string
    }
],
"meta": {
    "total": integer,
    "skip": integer,
    "take": integer
}
}

```

- **Error Responses:**

403 Forbidden

```

{
    "error_code": "FORBIDDEN",
    "message": "User does not have access to this building"
}

```

500 Internal Server Error

```

{
    "error_code": "SERVER_ERROR",
    "message": "Internal server error"
}

```

- **Effect:**

- Fetches anomalies associated with a specific building.

2. updateAnomalyStatus

- PATCH

- **Request:**

Path Parameters:

```
id: string
```

PATCH /api/anomalies/{id}/status

```

{
    "status": "Resolved"
}

```

- **Response: (200 OK)**

```

{
    "status": "success",
    "data": {
        "anomaly_id": string,
        "status": string,
        "resolved_timestamp": string
    }
}

```

```
}  
}
```

- **Error Responses:**

- 400 Bad Request**

```
{  
  "error_code": "INVALID_INPUT",  
  "message": "Invalidstatusvalueprovided"  
}
```

- 403 Forbidden**

```
{  
  "error_code": "FORBIDDEN",  
  "message": "User does not have access to this building"  
}
```

- 404 Not Found**

```
{  
  "error_code": "NOT_FOUND",  
  "message": "Anomaly not found"  
}
```

- 500 Internal Server Error**

```
{  
  "error_code": "SERVER_ERROR",  
  "message": "Internal server error"  
}
```

- **Effect:**

- Modifies the status of an anomaly and updates the resolved timestamp when it is marked as Resolved.

3. `getAnomalyContext`

- GET

- **Request:**

Path Parameters:
 id: string

GET /api/anomalies/{id}/context

- **Response: (200 OK)**

```
{  
  "status": "success",  
  "data": {  
    "anomaly_id": string,  
    "sensor": { ... },
```

```

        "threshold": { ... }
    }
}

```

- **Error Responses:**

403 Forbidden

```

{
  "error_code": "FORBIDDEN",
  "message": "User does not have access to this building"
}

```

404 Not Found

```

{
  "error_code": "NOT_FOUND",
  "message" : "Anomaly not found"
}

```

500 Internal Server Error

```

{
  "error_code": "SERVER_ERROR",
  "message" : "Internal server error"
}

```

- **Effect:**

- Fetches context for an anomaly, including the sensor data and the threshold that was breached.

5 Deployment

Deployment refers to publishing and running the software system in an accessible environment so that users, clients, and evaluators can interact with it without requiring local installation.

5.1 Deployment Requirements

5.1.1 Live, Accessible System

OptiGrid is deployed and accessible through the link in the root README. The frontend is accessible through a public URL and connects to the backend hosted on AWS. The AWS backend is only directly accessible to team members.

5.1.2 Environment Parity

The system distinguishes between development and production environments. The main branch automatically deploys to at least one non-local environment using the Vercel configuration.

Development is managed locally through Docker commands and the Supabase CLI or Supabase web editor to replicate the production database. Production is automatically deployed when the develop branch is merged into the main branch.

5.1.3 Infrastructure as Code (IaC) / Containerisation

The complete system is deployable through Docker and Docker Compose, ensuring deployments are reproducible across environments. No manual deployment steps, or “click-ops”, are required.

All backend and frontend services are containerised using optimised Dockerfiles in their respective folders. AWS resources are provisioned and controlled using Terraform.

5.1.4 Secrets Management

Environment-variable files are excluded through `.gitignore`. Secrets are stored securely using GitHub Secrets and Vercel or AWS Parameter Store.

5.1.5 Rollback Strategy

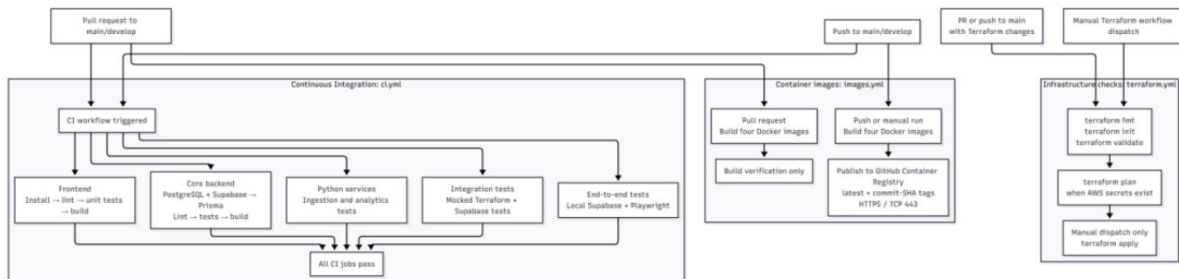
To ensure rapid recovery from deployment failure, the system implements a multi-tiered rollback strategy:

1. **Container Image Tagging:** Production Docker images are tagged with their release commit hash as well as `latest`. If a deployment fails, the containers can be reverted to the previously working version.
2. **Frontend Instant Rollback:** Vercel deployments support immediate rollback to the last known stable frontend deployment.

5.2 Deployment Diagrams

The following diagrams illustrate where the system is deployed and how software changes move through the deployment pipeline.

5.2.1 CI/CD Pipeline Diagram



6 NFR Traceability Matrix

The following matrix maps each tested Non-Functional Requirement to its architectural tactic and test evidence. The recorded results are taken from the executed Security, Performance, Availability, Reliability, and Maintainability test suites.

Table 2: Complete NFR Traceability Matrix

ID	Quantified Requirement	Tactic in SAS	Test / Evidence	Status
Security Tests				
SEC01	No critical or high-severity vulnerabilities may exist in production dependencies.	Automated dependency auditing.	Run <code>pnpm audit --json --prod</code> . No critical or high-severity vulnerabilities were reported.	PASS
SEC02	Backend responses must include HSTS, CSP, and X-Frame-Options security headers.	Security-header middleware.	Inspect the HTTP headers returned by the <code>/health</code> endpoint.	PASS
SEC03	Automated request bursts must be rate-limited to reduce abuse and brute-force risk.	API rate limiting.	Send 150 requests and verify that the API responds with HTTP 429 Too Many Requests.	PASS
SEC04	Authentication flows and sensitive information must use the required security controls.	JWT authentication and AES-256 encryption.	Execute authentication tests and inspect the cryptographic implementation.	PASS
SEC05	Protected resources must not be accessible without a valid JSON Web Token.	Authentication and route-protection middleware.	Request a protected endpoint without a valid token and verify HTTP 401.	PASS
SEC06	CORS policies must reject requests from unknown or malicious origins.	Restricted CORS configuration.	Submit a request using <code>Origin: malicious.com</code> and verify rejection.	PASS
SEC07	Administrative endpoints must enforce role-based permission boundaries.	Role-verification middleware and RBAC.	Inspect and test route-level permissions on administrative endpoints.	PASS
Performance Tests				
PERF-01	Smoke test with 1 virtual user for 30 seconds. The p95 response time must remain below 2000 ms and the error rate below 1%.	Non-blocking request handling.	The k6 smoke test completed 30 iterations without interrupted iterations. The response times remained below the two-second threshold.	PASS
PERF-02	Concurrency test with 50 virtual users over 3 minutes. The p95 response time must remain below 2000 ms and the error rate below 1%.	Concurrent request handling and load balancing.	Run the k6 concurrency scenario and verify the configured response-time and failure-rate thresholds.	PASS

Continued on next page

Table 2 – continued from previous page

ID	Quantified Requirement	Tactic in SAS	Test / Evidence	Status
PERF-03	Load test with 500 virtual users over 5 minutes. The p95 response time must remain below 2000 ms and the error rate below 1%.	Load balancing and asynchronous processing.	Run the k6 load scenario and verify the configured response-time and failure-rate thresholds.	PASS
Availability Tests				
A01	The system must be available 24/7, excluding scheduled maintenance.	Health checks and service monitoring.	The core <code>/health</code> endpoint returned HTTP 200.	PASS
A02	Redundant components must fail smoothly without making the complete system unavailable.	Redundancy and managed restart.	An ingestion API container was paused while the core service remained available.	PASS
A03	The system must degrade gracefully during network or database latency.	Timeout handling and graceful degradation.	The database container was paused and the core service responded without hanging indefinitely.	PASS
A04	Deployment must support zero-downtime operation.	Rolling restarts and health verification.	The core health endpoint remained available during the simulated deployment validation.	PASS
A05	The ingestion API must be independently available.	Independent ingestion microservice.	The ingestion health endpoint returned HTTP 200.	PASS
A06	The analytics service must be independently available.	Independent analytics microservice.	The analytics health endpoint returned HTTP 200.	PASS
A07	The frontend application must be available.	Independent frontend deployment.	The frontend application returned HTTP 200.	PASS
A08	The core system must remain available when the analytics service fails.	Fault isolation between microservices.	The analytics container was paused while the core health endpoint continued to return HTTP 200.	PASS
Reliability Tests				
R01	The system must recover from critical failures within five minutes.	Managed restart and service-health verification.	Redis was paused and restored. The core service recovered successfully.	PASS
R02	The system must maintain 99.9% uptime under load.	Repeated health checks and stable request handling.	All sampled health requests returned HTTP 200.	PASS
R03	The system must recover after the database connection is interrupted.	Connection pooling and database reconnection.	PostgreSQL was paused and restored. The core health endpoint returned HTTP 200 after recovery.	PASS
R04	Malformed telemetry must not crash the ingestion API.	Input validation and defensive request parsing.	Malformed telemetry JSON produced a controlled error response without crashing the service.	PASS

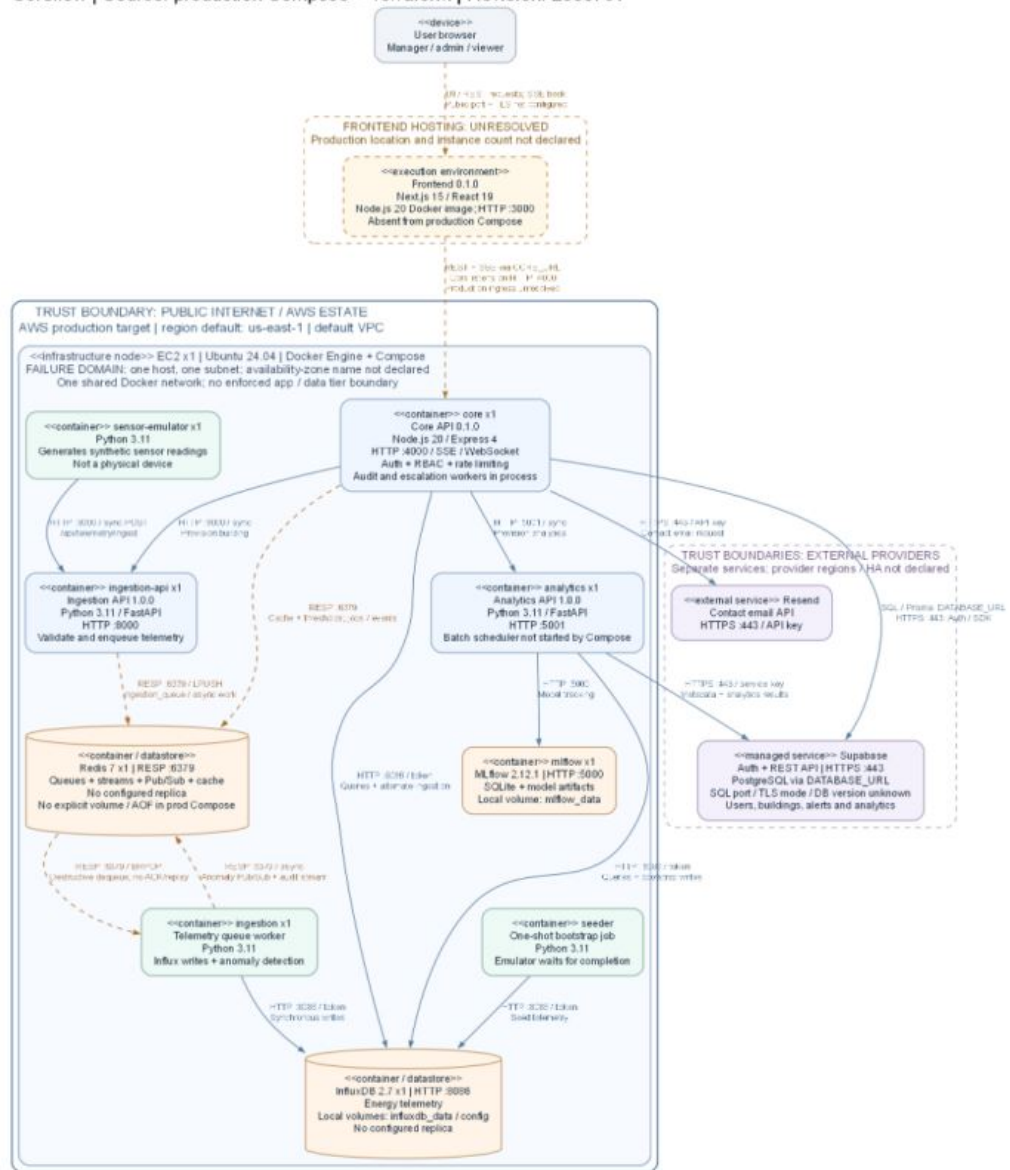
Continued on next page

Table 2 – continued from previous page

ID	Quantified Requirement	Tactic in SAS	Test / Evidence	Status
R05	Oversized telemetry payloads must be rejected safely.	Payload-size limits and request validation.	An oversized telemetry payload produced a controlled rejection response.	PASS
R06	The ingestion API must remain resilient while InfluxDB is unavailable.	Redis buffering, retries, and graceful failure handling.	InfluxDB was paused and telemetry was buffered or rejected safely.	PASS
R07	The core API must safely handle unauthenticated requests.	Authentication middleware and controlled error handling.	A protected endpoint returned HTTP 401 without destabilising the API.	PASS
R08	The system must recover from simultaneous service disruption.	Service isolation and container recovery.	Redis and InfluxDB were paused together and restored successfully.	PASS
Maintainability Tests				
MNT-01	Frontend automated test coverage must be at least 80%.	Jest unit tests and coverage reporting.	Frontend coverage was 87.53% : 4466 of 5102 measured lines were covered.	PASS
MNT-02	Backend core automated test coverage must be at least 80%.	Jest unit tests and coverage reporting.	Backend core coverage was 86.88% : 3737 of 4301 measured lines were covered.	PASS
MNT-03	Ingestion-service automated test coverage must be at least 80%.	pytest and pytest-cov coverage reporting.	Ingestion coverage was 86.34% : 373 of 432 measured statements were covered.	PASS
MNT-04	Analytics-service automated test coverage must be at least 80%.	pytest and pytest-cov coverage reporting.	Analytics coverage was 85.12% : 732 of 860 measured statements were covered.	PASS
MNT-B05	TypeScript modules must comply with the documented architectural layer boundaries.	Automated dependency-boundary analysis.	The TypeScript boundary checker found no violations across 250 files .	PASS
MNT-06	Python modules must comply with the documented architectural layer boundaries.	Automated Python dependency analysis.	The Python boundary checker found no violations across 13 Python files .	PASS
MNT-07	The repository must pass the configured code-quality and linting rules.	ESLint static analysis.	ESLint completed with 0 errors across the repository.	PASS
MNT-08	Combined system test coverage must be at least 80%.	Aggregated frontend, backend, ingestion, and analytics coverage reporting.	Overall coverage was 87.03% : 9308 of 10695 measured lines or statements were covered.	PASS

7 Deployment Diagram

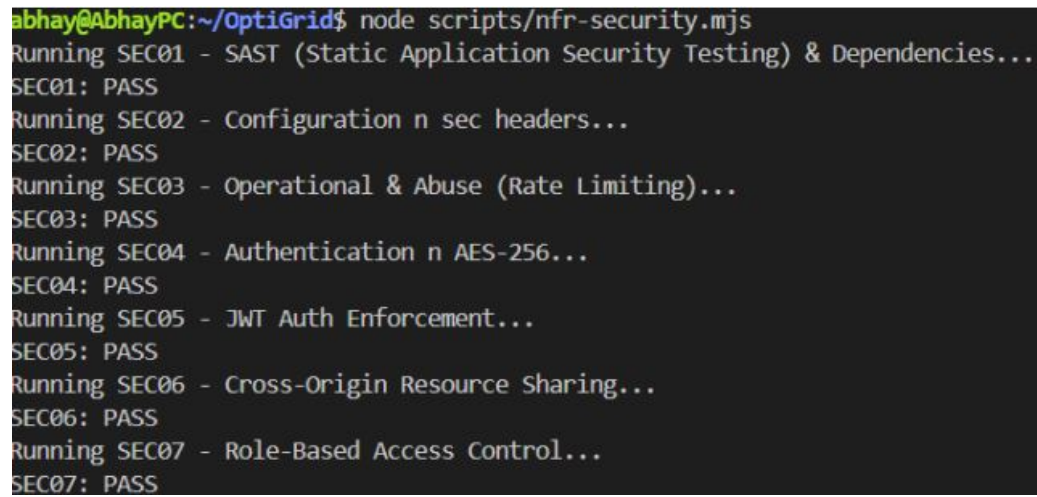
OptiGrid | Deployment diagram
 Configured production target / repository evidence, 03 Sep 2026
 Coreflow | Source: production Compose + Terraform | Revision: 26037c1



8 Test Evidence (Screenshots)

This section contains visual evidence of the executed NFR tests.

8.1 Security Tests



```
abhay@AbhayPC:~/OptiGrid$ node scripts/nfr-security.mjs
Running SEC01 - SAST (Static Application Security Testing) & Dependencies...
SEC01: PASS
Running SEC02 - Configuration n sec headers...
SEC02: PASS
Running SEC03 - Operational & Abuse (Rate Limiting)...
SEC03: PASS
Running SEC04 - Authentication n AES-256...
SEC04: PASS
Running SEC05 - JWT Auth Enforcement...
SEC05: PASS
Running SEC06 - Cross-Origin Resource Sharing...
SEC06: PASS
Running SEC07 - Role-Based Access Control...
SEC07: PASS
```

8.2 Performance Tests

```

abhay@AbhayPC:~/OptiGrid$ node scripts/nfr-performance.mjs
Running k6 scenario: smoke...
Test evidence will be saved to: /home/abhay/OptiGrid/test-results/performance/2026-09-29T17-59-30-932Z

  A  K6  .io

execution: local
  script: /home/abhay/OptiGrid/test-results/performance/2026-09-29T17-59-30-932Z/k6-test.js
  output: -

scenarios: (100.00%) 1 scenario, 1 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 1 looping VUs for 30s (gracefulStop: 30s)

✓ health returned 200
✓ health answered within 2s

checks.....: 100.00% ✓ 60      ✗ 0
data_received.....: 14 kB    461 B/s
data_sent.....: 3.5 kB    120 B/s
http_req_blocked.....: avg=8.69µs   min=3.11µs   med=3.83µs   max=135.73µs p(90)=5.6µs p(95)=10.55µs
http_req_connecting.....: avg=2.6µs    min=0s       med=0s       max=78.02µs p(90)=0s   p(95)=0s
✓ http_req_duration.....: avg=1.07ms   min=757.96µs med=881.38µs max=5.03ms p(90)=1.25ms p(95)=1.54ms
  { expected response:true }...: avg=1.07ms   min=757.96µs med=881.38µs max=5.03ms p(90)=1.25ms p(95)=1.54ms
✓ http_req_failed.....: 0.00% ✓ 0      ✗ 30
http_req_receiving.....: avg=67.74µs  min=37.98µs  med=48.91µs  max=369.69µs p(90)=91.88µs p(95)=122.55µs
http_req_sending.....: avg=22.35µs  min=12.7µs   med=23.35µs  max=57.06µs p(90)=28.01µs p(95)=33.21µs
http_req_tls_handshaking.....: avg=0s       min=0s       med=0s       max=0s p(90)=0s   p(95)=0s
http_req_waiting.....: avg=981.03µs min=691.77µs med=785.24µs max=4.97ms p(90)=1.12ms p(95)=1.48ms
http_reqs.....: 30    1.017106/s
iteration_duration.....: avg=1s       min=1s       med=1s       max=1s p(90)=1s   p(95)=1s
iterations.....: 30    1.017106/s
vus.....: 1    min=1    max=1
vus_max.....: 1    min=1    max=1

running (0m29.5s), 0/1 VUs, 30 complete and 0 interrupted iterations
default ✓ [=====] 1 VUs 30s

```

8.3 Availability Tests

```
abhay@AbhayPC:~/OptiGrid$ node scripts/nfr-availability.mjs
Running A01 - Available 24/7, excluding maintenance...
A01: PASS
Running A02 - Redundant components fail smoothly...
A02: PASS
Running A03 - Graceful degradation under network latency...
A03: PASS
Running A04 - Zero-downtime deployment...
A04: PASS
Running A05 - Ingestion API is independently available...
A05: PASS
Running A06 - Analytics Service is independently available...
A06: PASS
Running A07 - Frontend application is available...
A07: PASS
Running A08 - System remains available when analytics fails...
A08: PASS
```

8.4 Reliability Tests

```
abhay@AbhayPC:~/OptiGrid$ node scripts/nfr-reliability.mjs
Running R01 - Recover from critical failures within 5 minutes...
R01: PASS
Running R02 - 99.9% uptime under load...
R02: PASS
Running R03 - Database connection recovery...
R03: PASS
Running R04 - Resilience to malformed telemetry data...
R04: PASS
Running R05 - Resilience to oversized telemetry payloads...
R05: PASS
Running R06 - Ingestion API resilience to InfluxDB downtime...
R06: PASS
Running R07 - Core API resilience to unauthenticated requests...
R07: PASS
Running R08 - System recovers from simultaneous service disruption...
R08: PASS
```

8.5 Maintainability Tests

```
● hamdaan_mirza@DESKTOP-SCQ7364:~/OptiGrid$ node scripts/nfr-maintainability.mjs
Running MNT-01 - Frontend Test Coverage...
MNT-01: PASS - Coverage is 87.53% (4466/5102 lines)
Running MNT-02 - Backend Core Test Coverage...
MNT-02: PASS - Coverage is 86.88% (3737/4301 lines)
Running MNT-03 - Ingestion Test Coverage...
MNT-03: PASS - Coverage is 86.34% (373/432 statements)
Running MNT-04 - Analytics Test Coverage...
MNT-04: PASS - Coverage is 85.12% (732/860 statements)
Running MNT-B05 - Layer Boundaries(TypeScript)...
MNT-B05: PASS - PASS: NO violations out of 250 files
Running MNT-06 - Layer Boundaries (Python)...
MNT-06: PASS - PASS: NO violations out of 13 python files
Running MNT-07 - Code Quality & Linting...
MNT-07: PASS - PASS: ESLint has 0 errors across the repo
Running MNT-08 - Overall System Test Coverage...
MNT-08: PASS - Overall coverage is 87.03% (9308/10695 lines/statements)
```